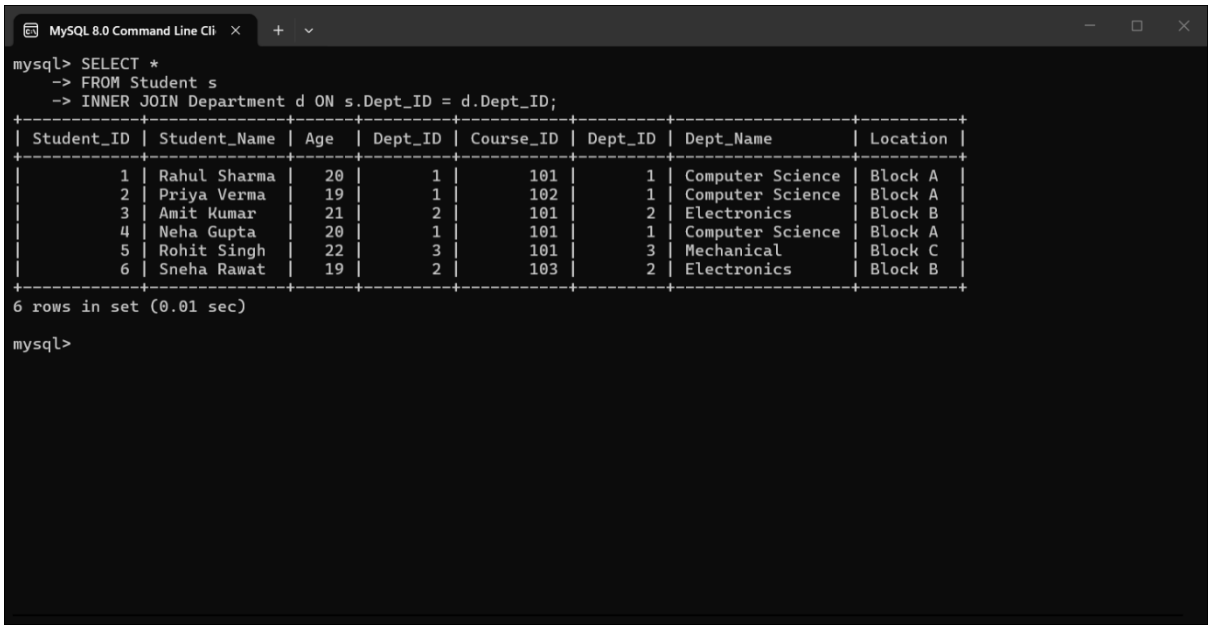
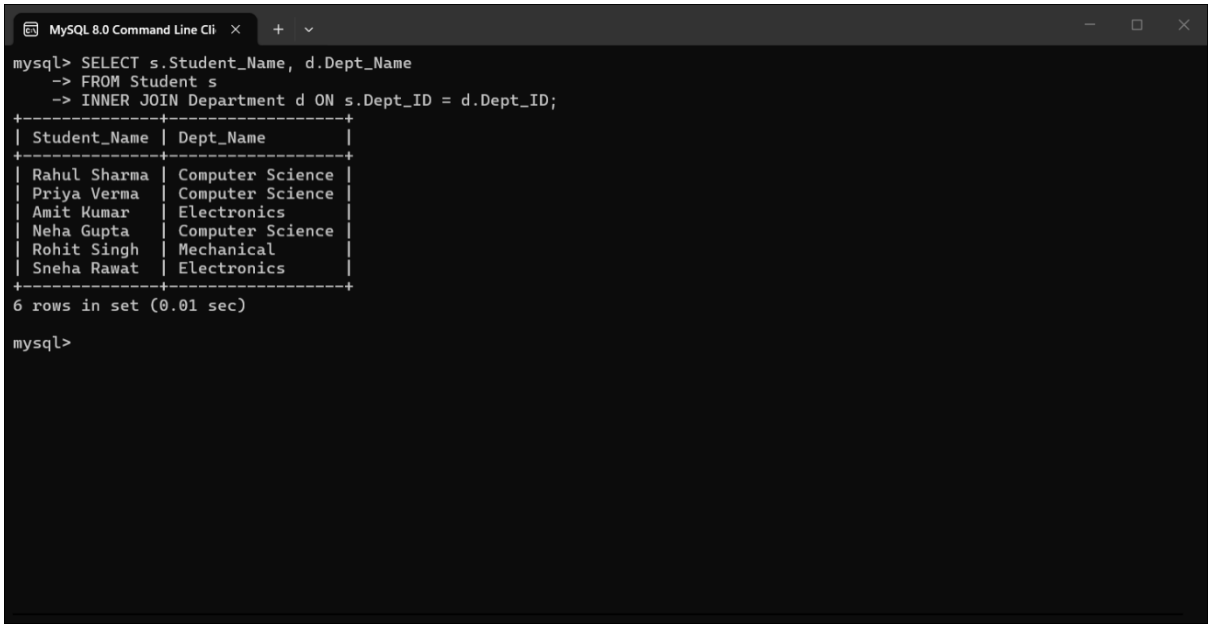
Lab Sheet 5

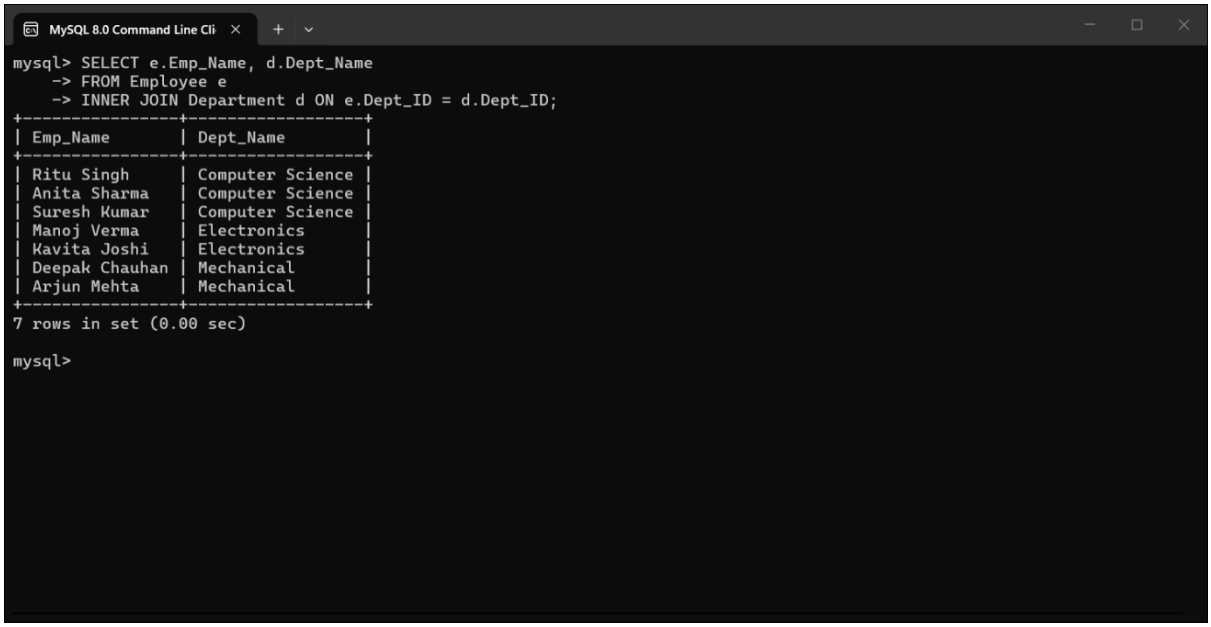
1. Demonstrate an INNER JOIN between Student and Department



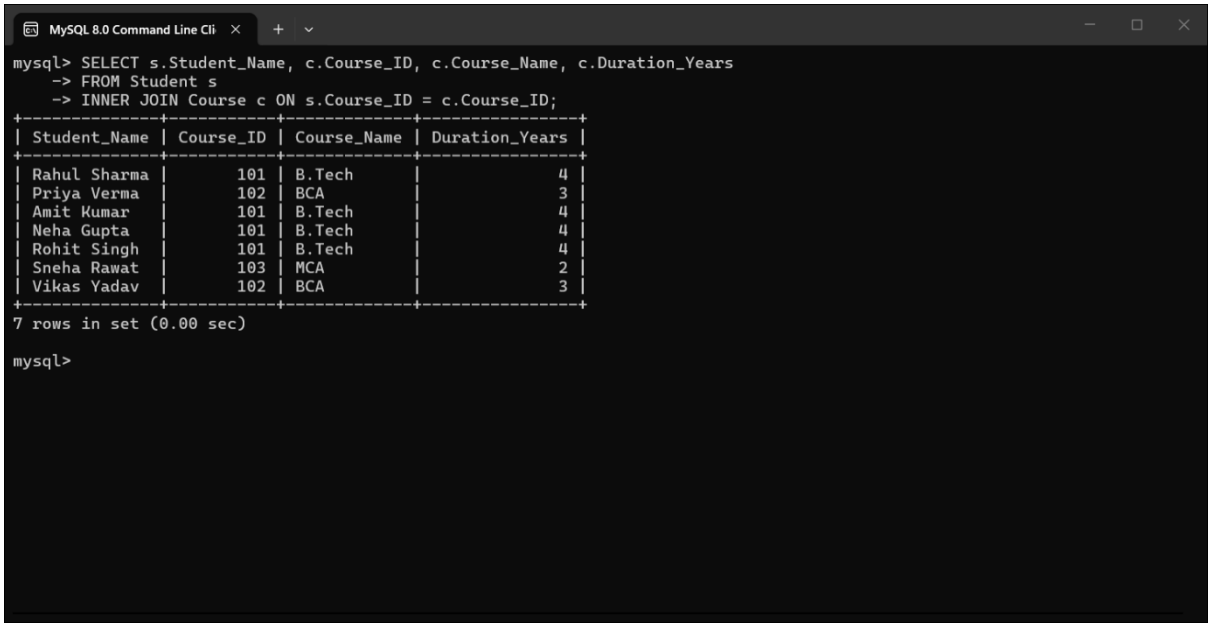
2. Display student names along with their department names using INNER JOIN.



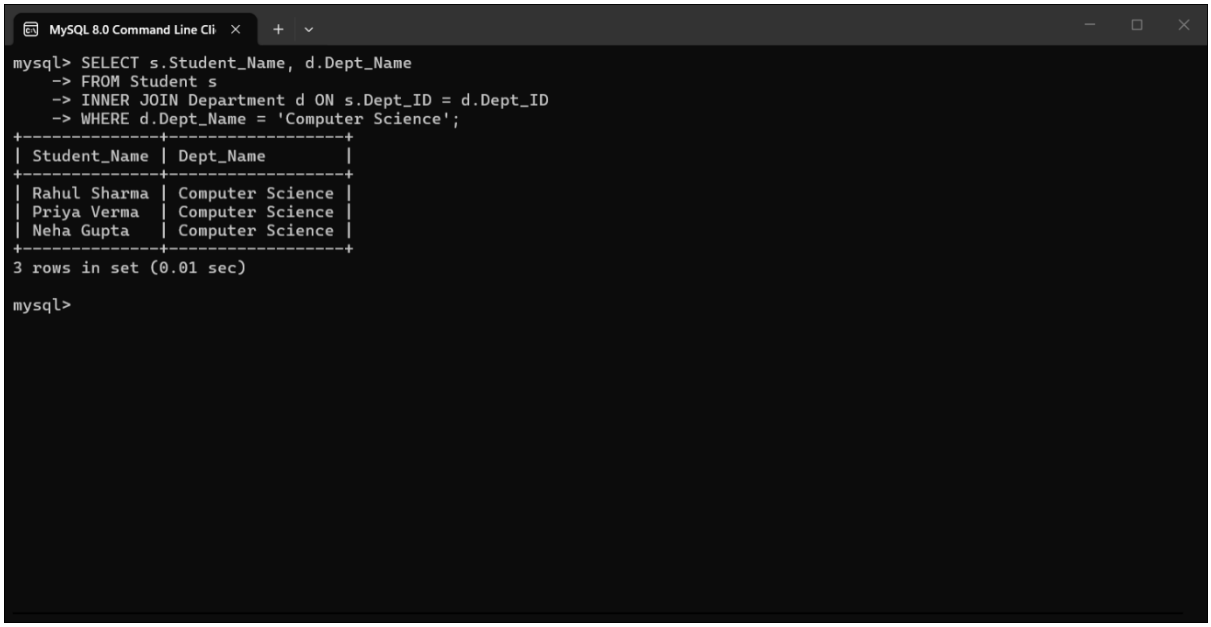
3. Display employee names along with their department names.



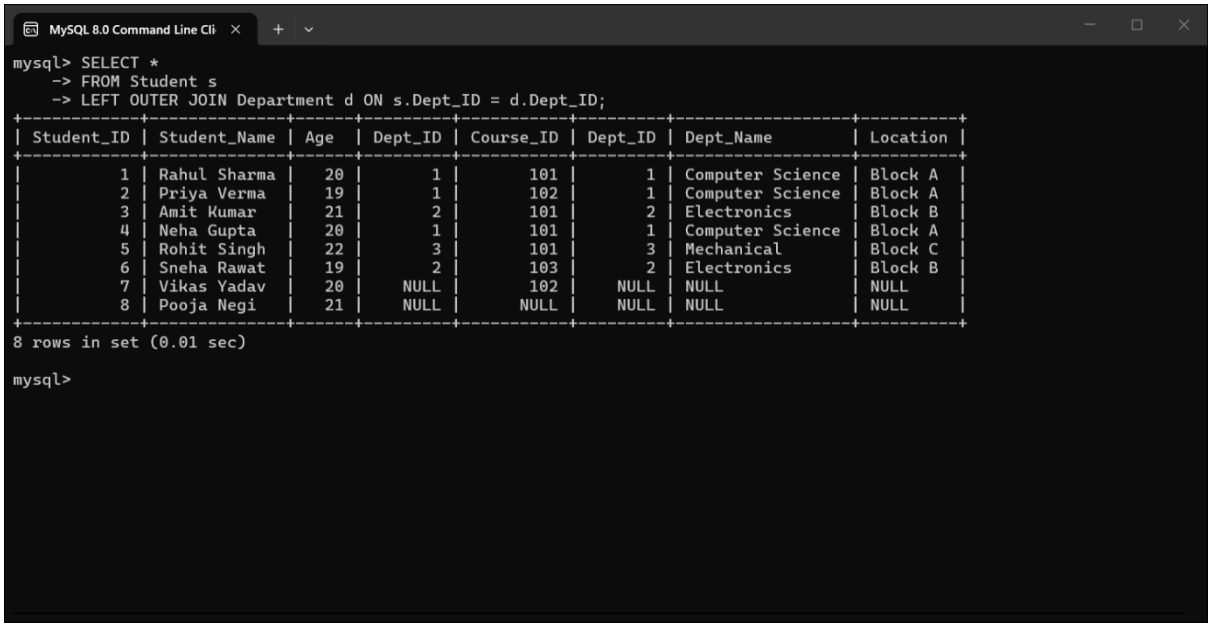
4. Display students along with their course details.



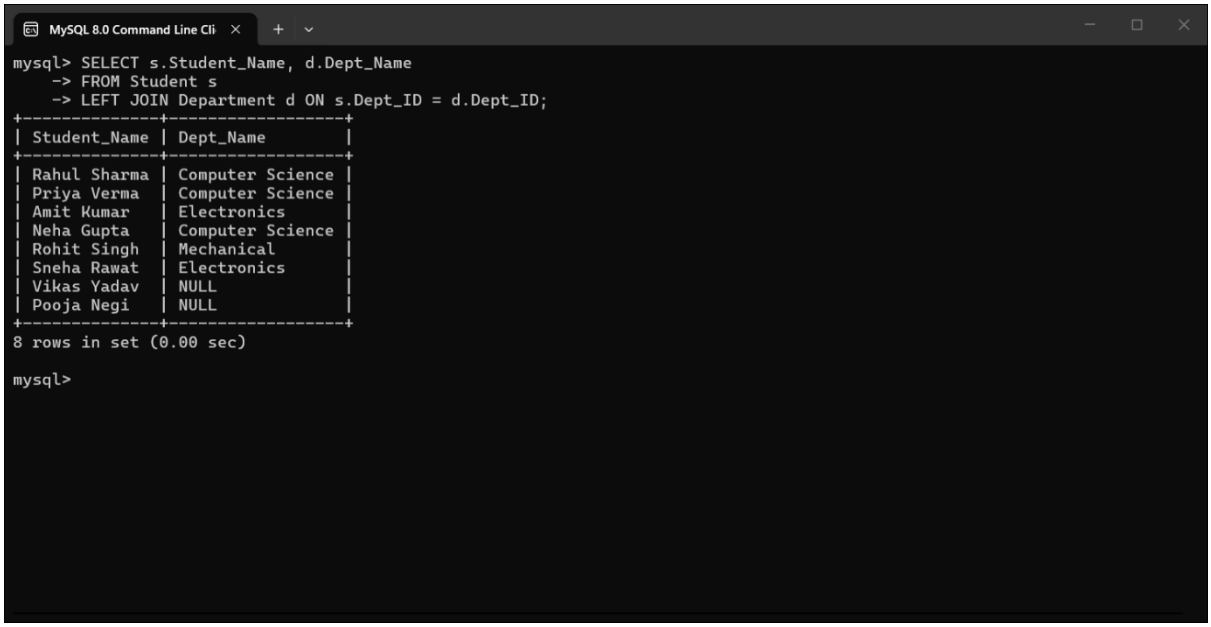
5. Find students belonging to a particular department using INNER JOIN.



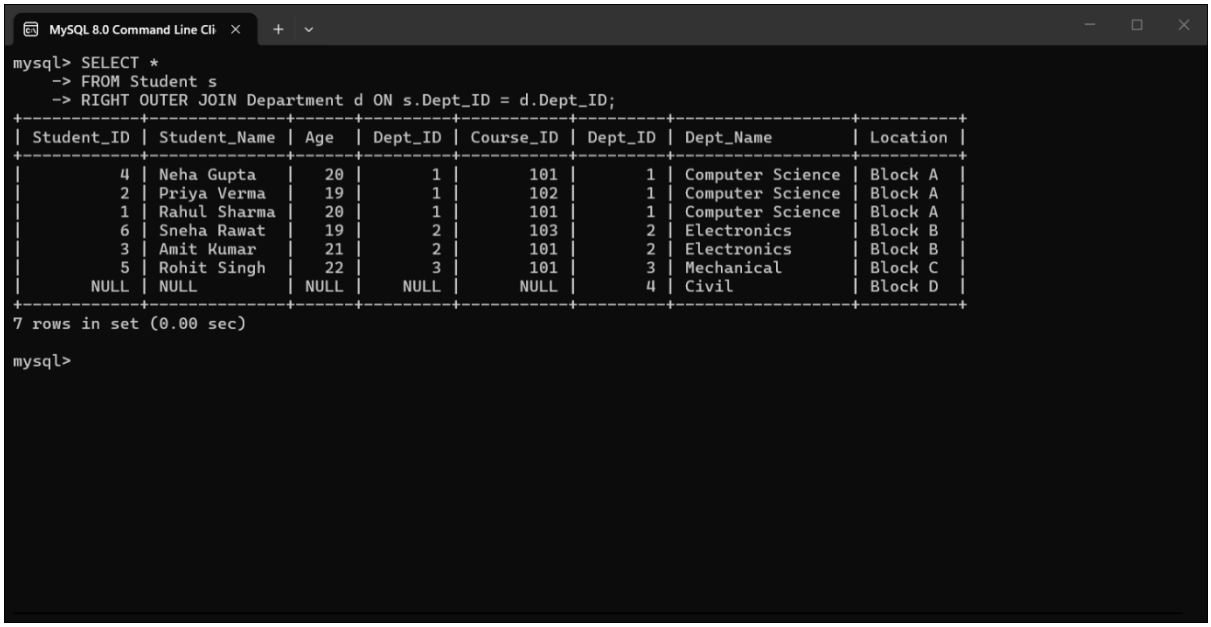
6. Demonstrate a LEFT OUTER JOIN between Student and Department.



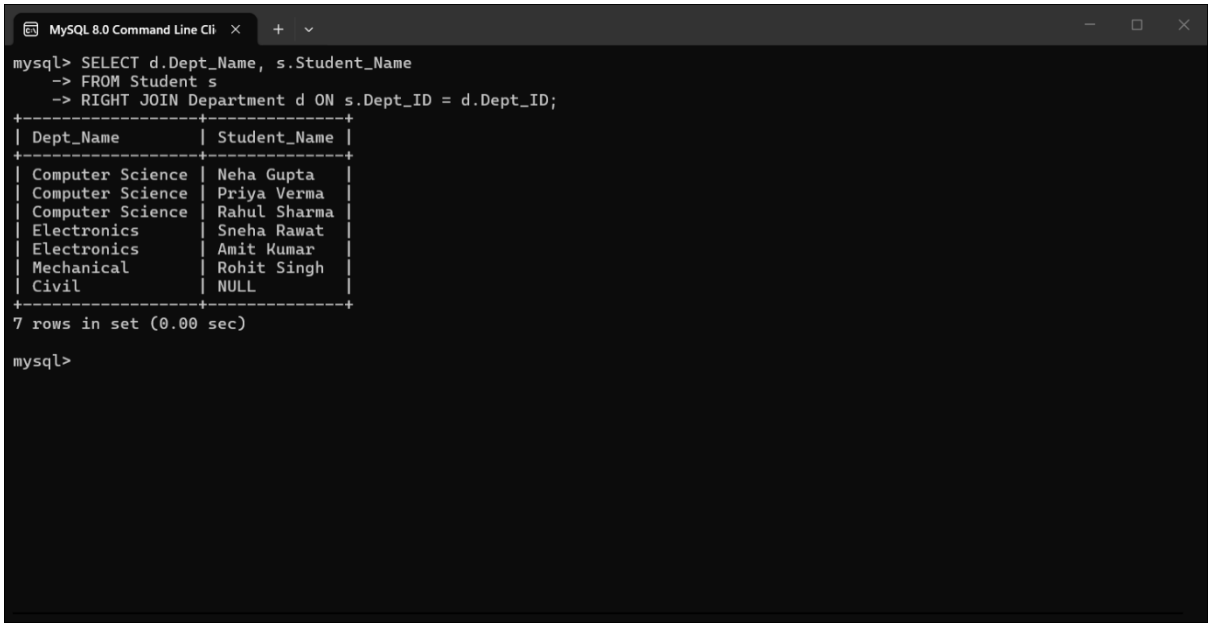
7. Display all students, including those who do not have a matching department.



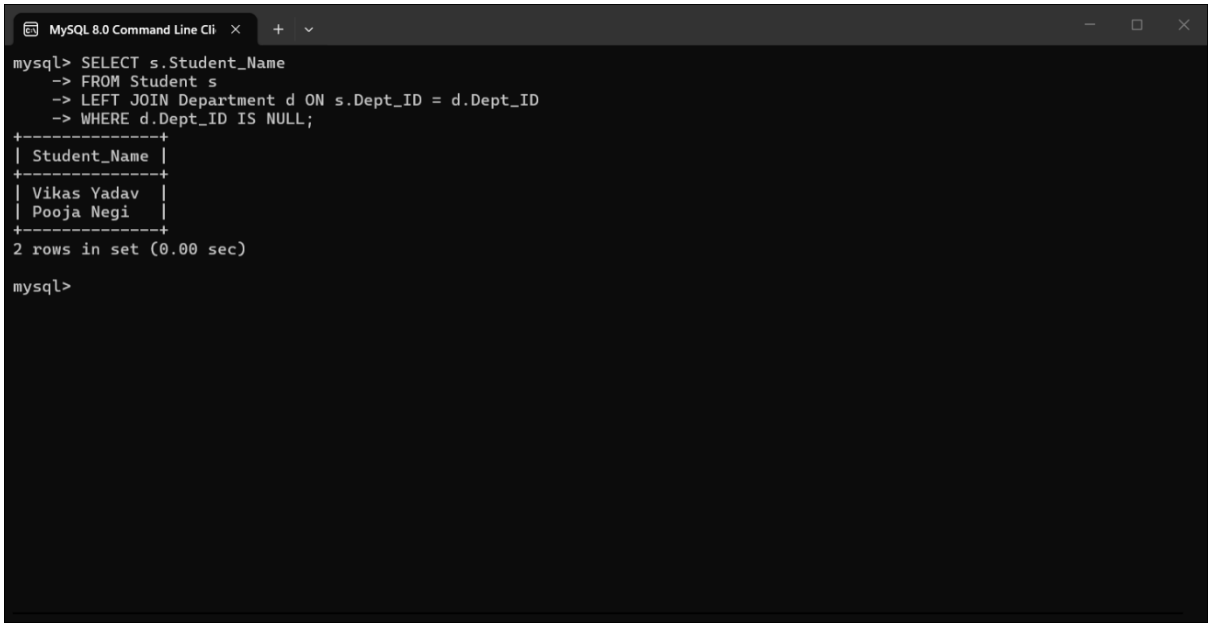
8. Demonstrate a RIGHT OUTER JOIN between Student and Department.



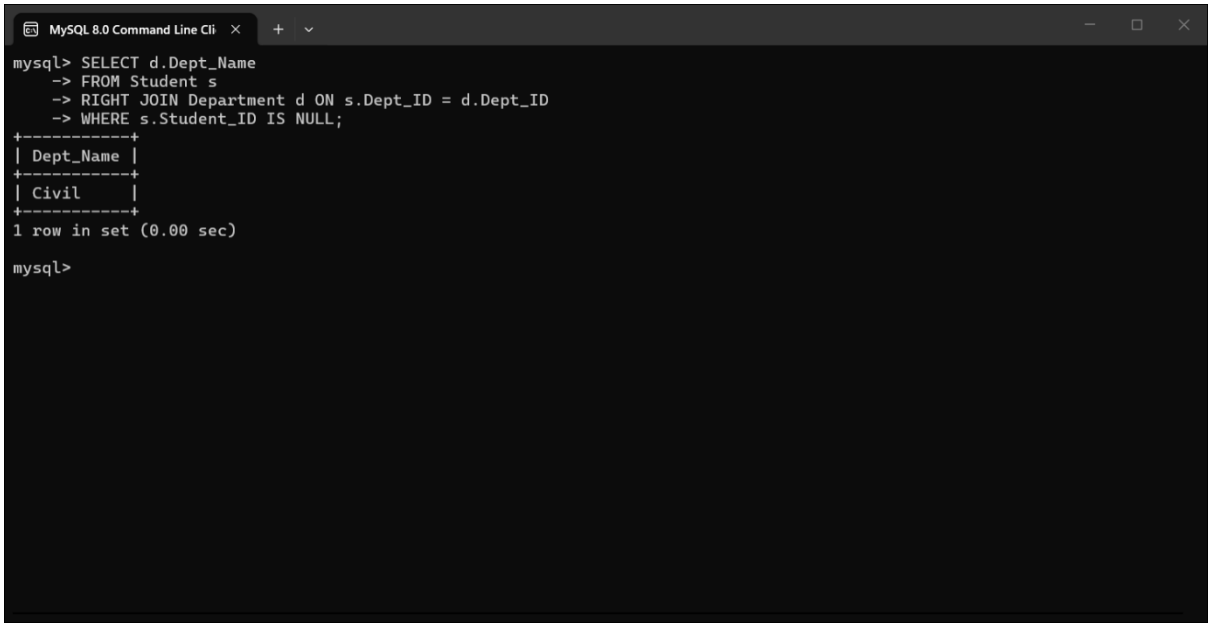
9. Display all departments, including departments having no students.



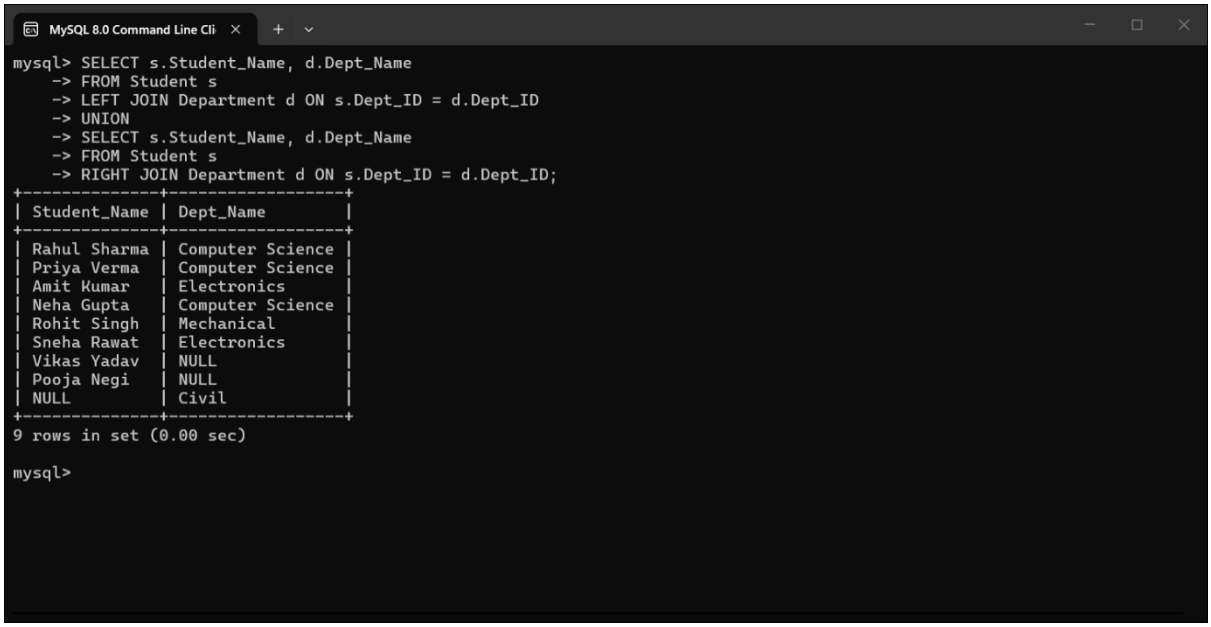
10. Find students who are not assigned to any department using LEFT JOIN.



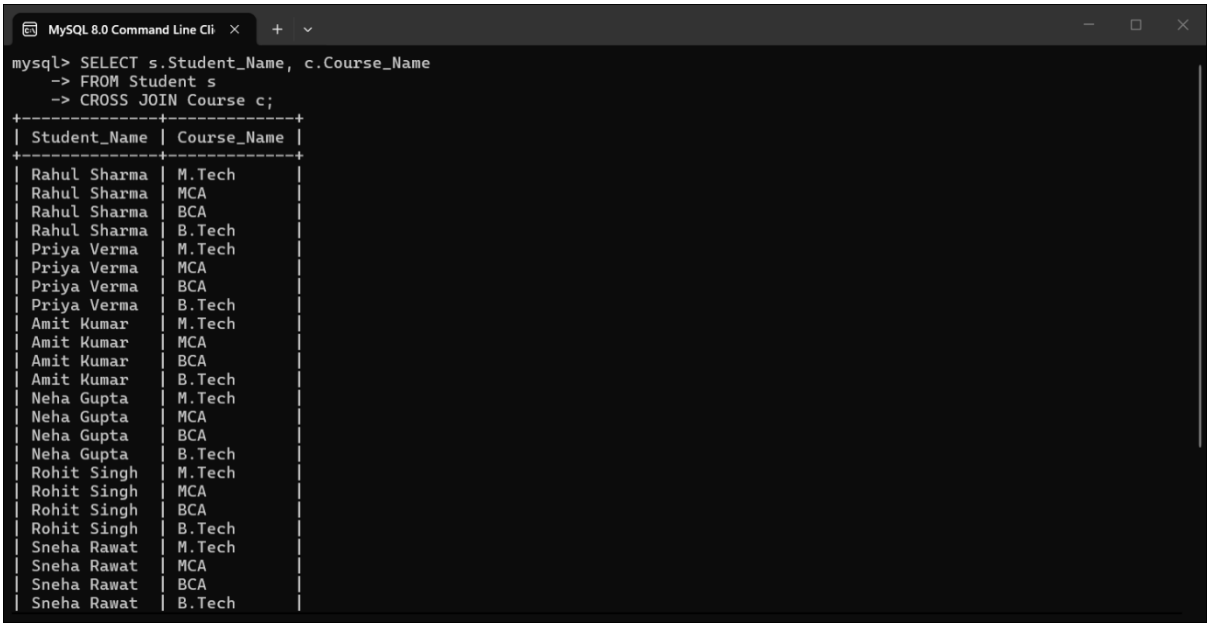
11. Find departments having no students using RIGHT/LEFT JOIN.

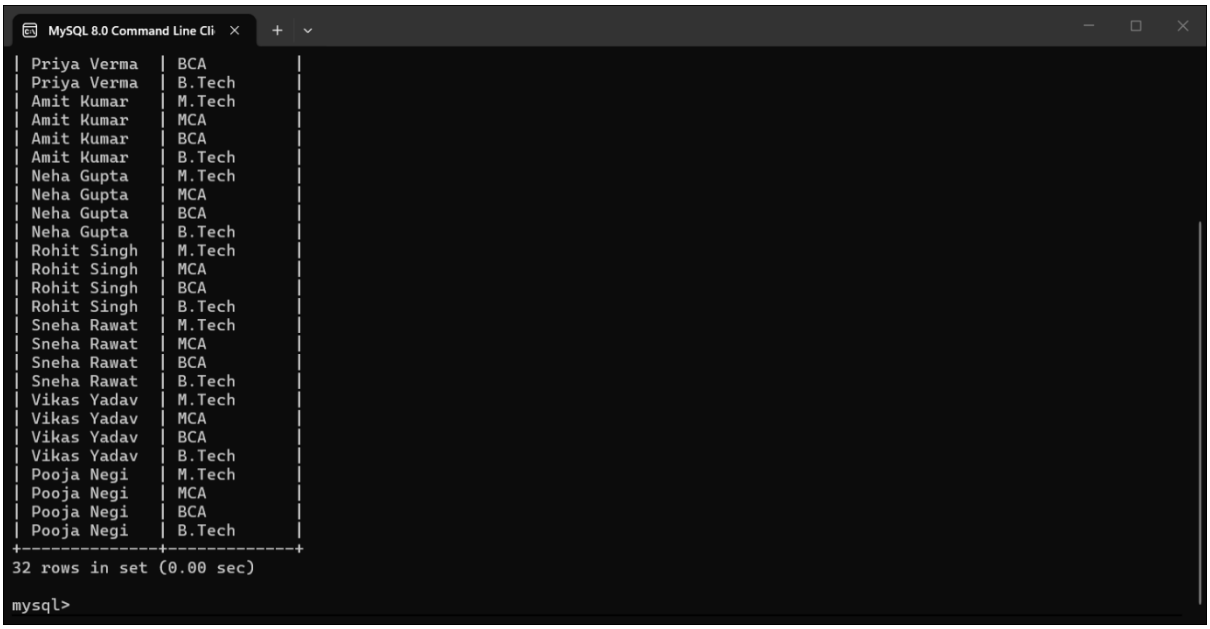


12. Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround where supported.

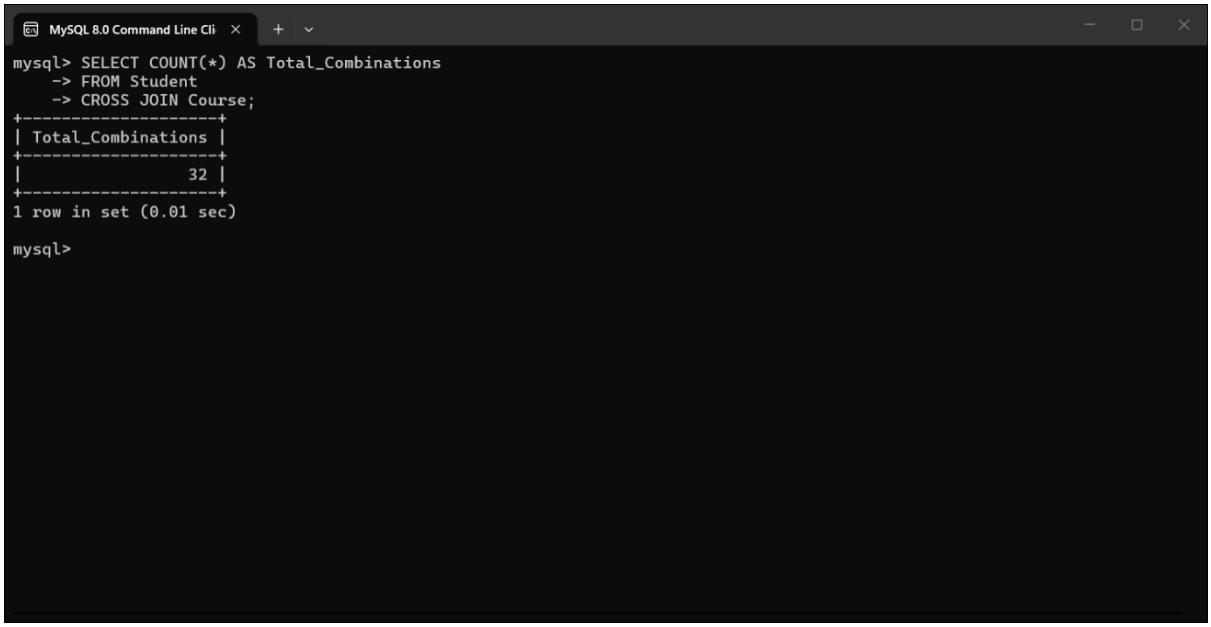


13. Demonstrate a CROSS JOIN between Student and Course.

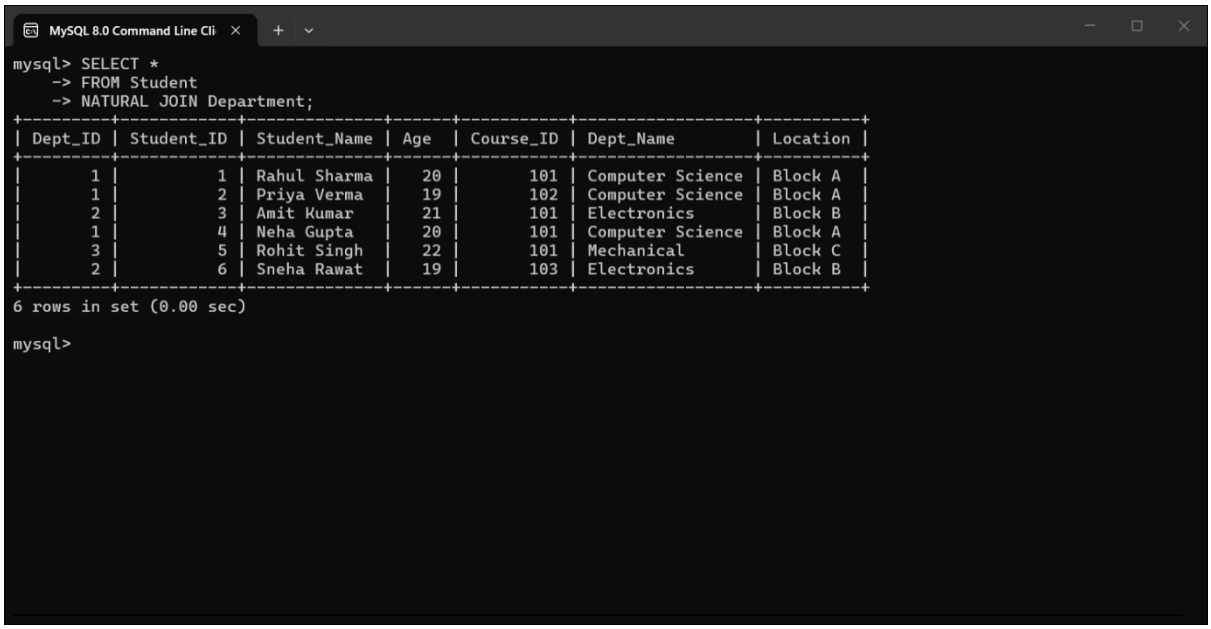




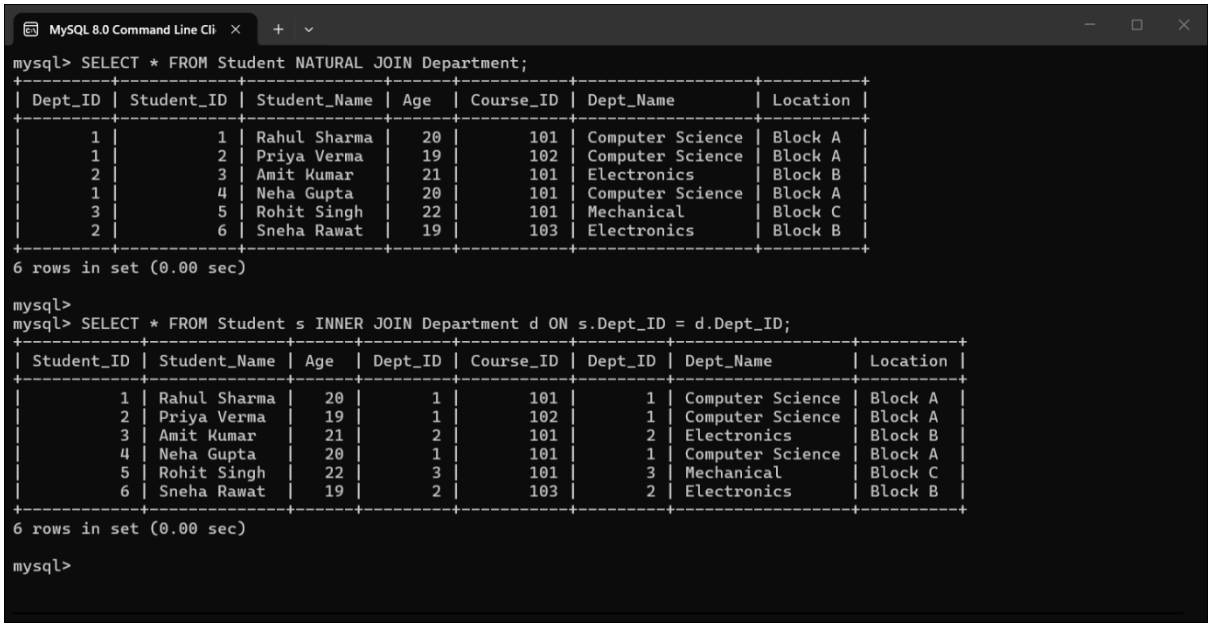
14. Calculate the number of possible Student-Course combinations using CROSS JOIN.
    Number of possible Student-Course pairs (8 students x 4 courses = 32)



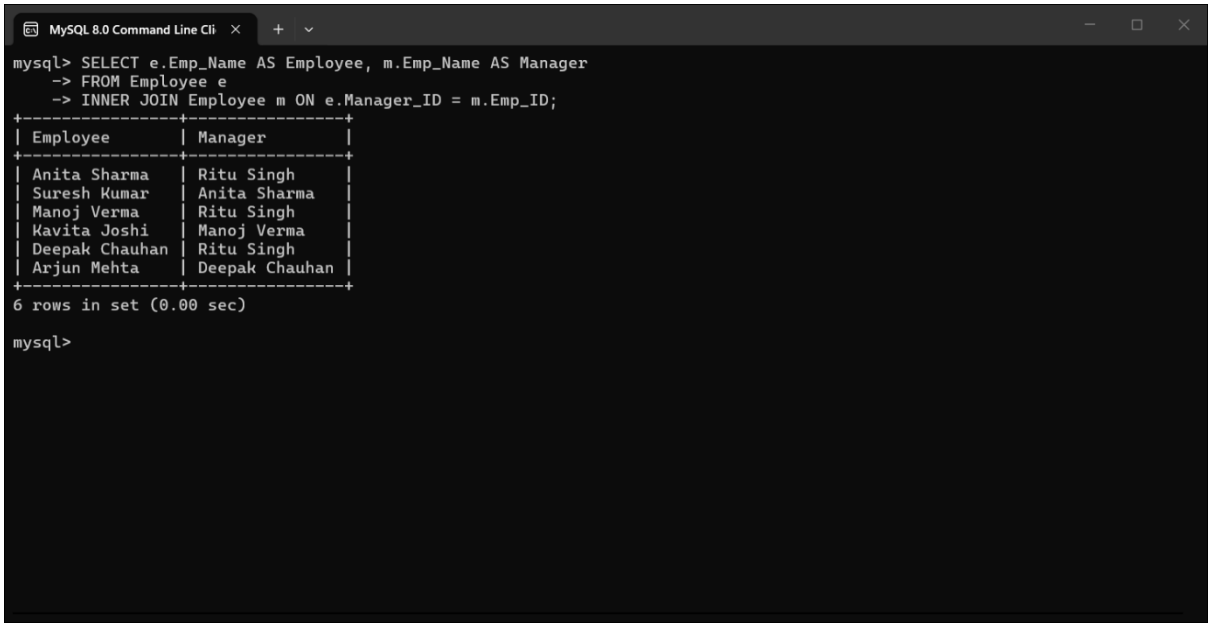
15. Demonstrate a NATURAL JOIN between two suitable tables.
    NATURAL JOIN : joins on columns having the same name(here the common column is Dept_ID)



16. Compare the result of NATURAL JOIN and INNER JOIN



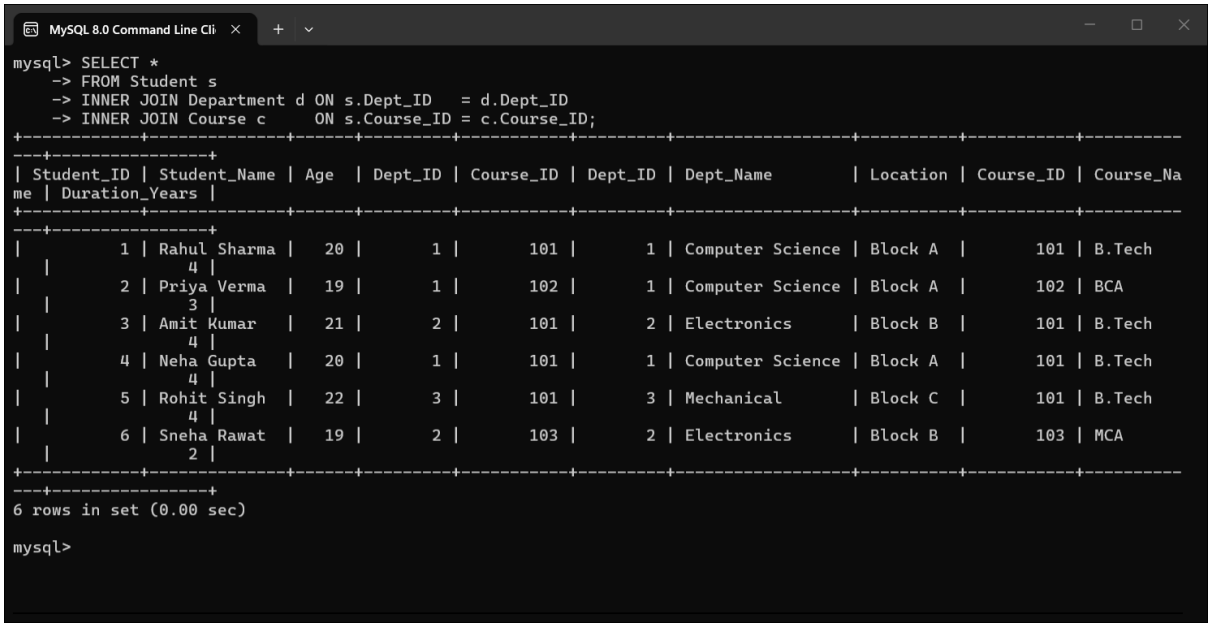
17. Perform a SELF JOIN on an Employee table to show employee-manager relationships.



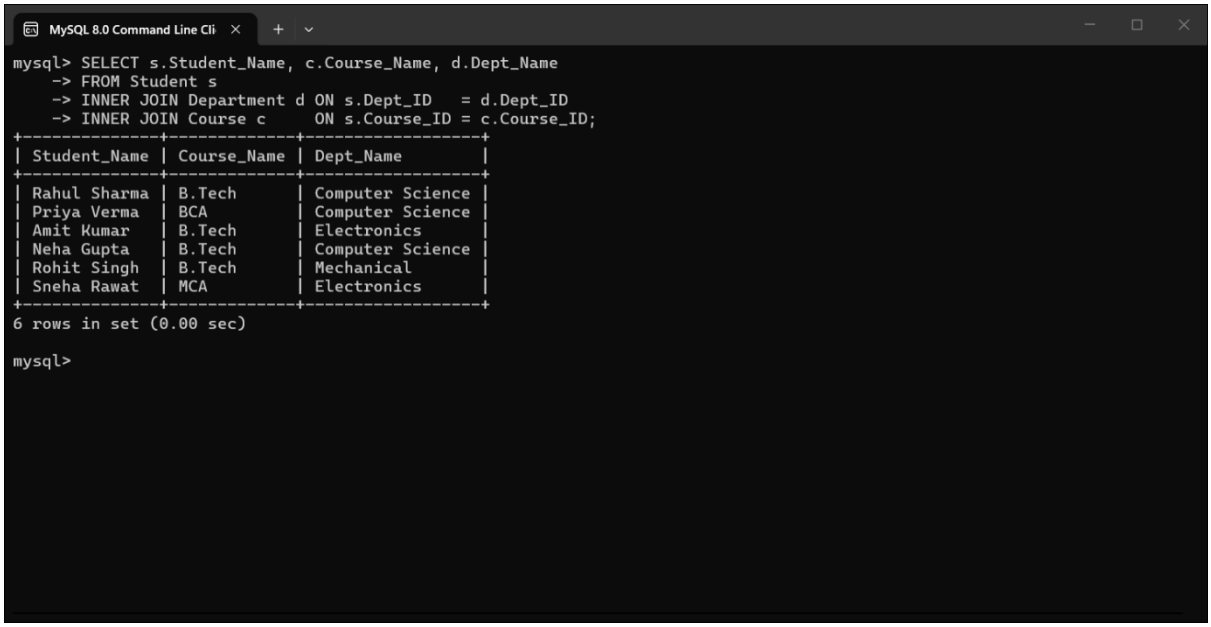
18. Display employees along with their managers using SELF JOIN.



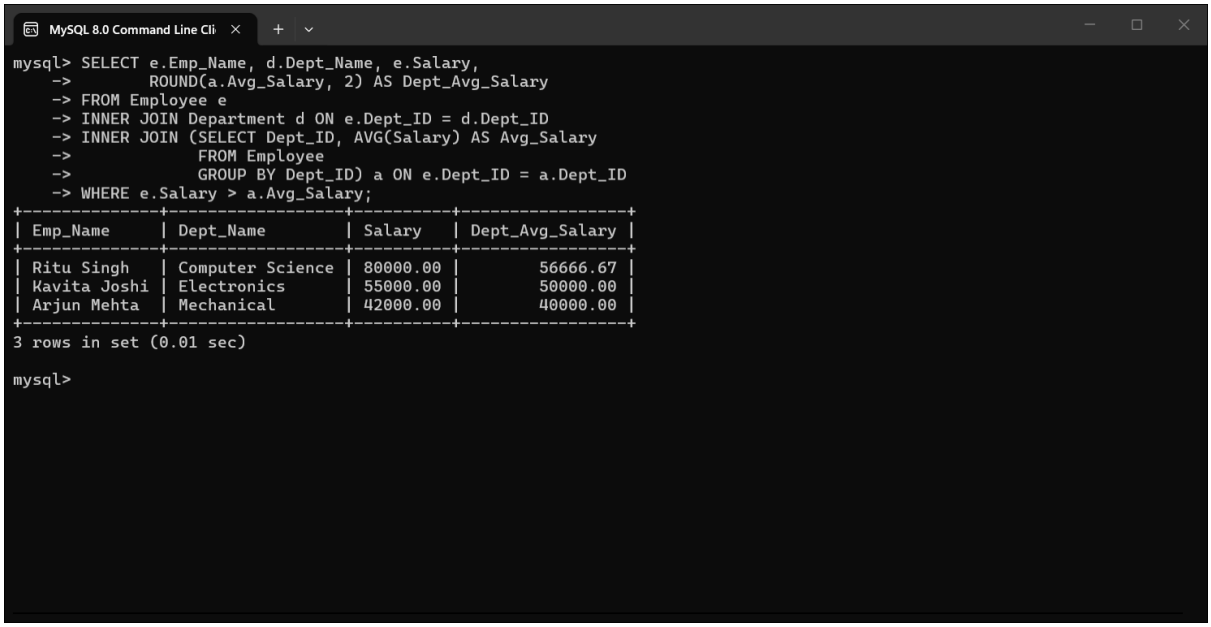
19. Join three tables—Student, Department and Course.



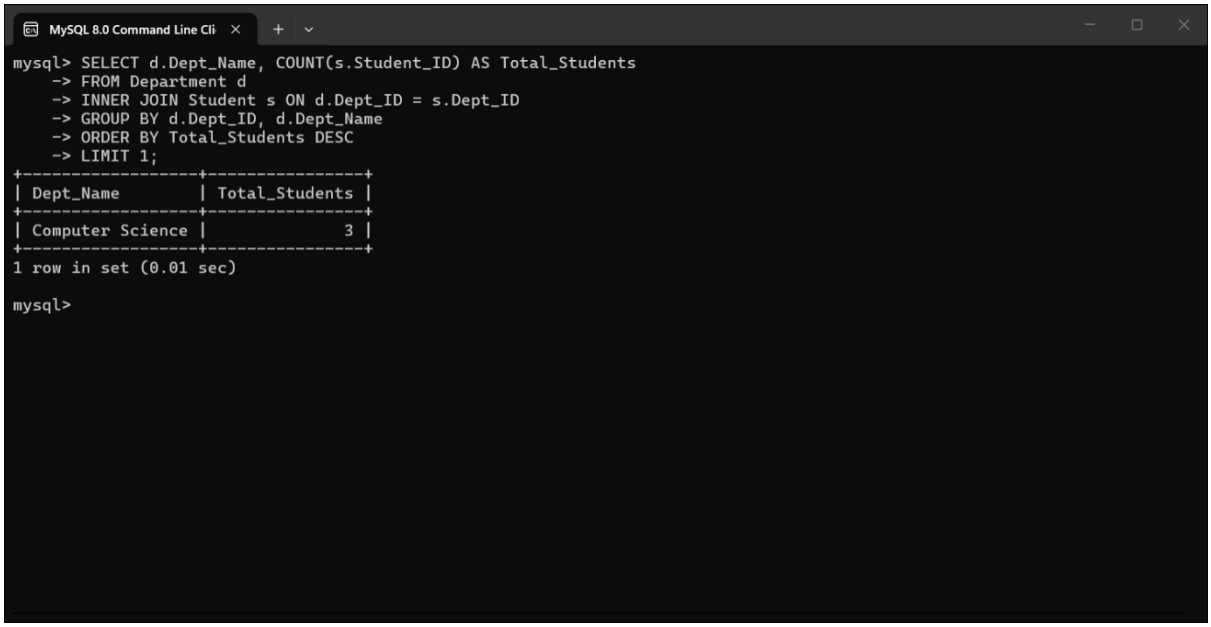
20. Display student name, course name and department name using multiple joins.



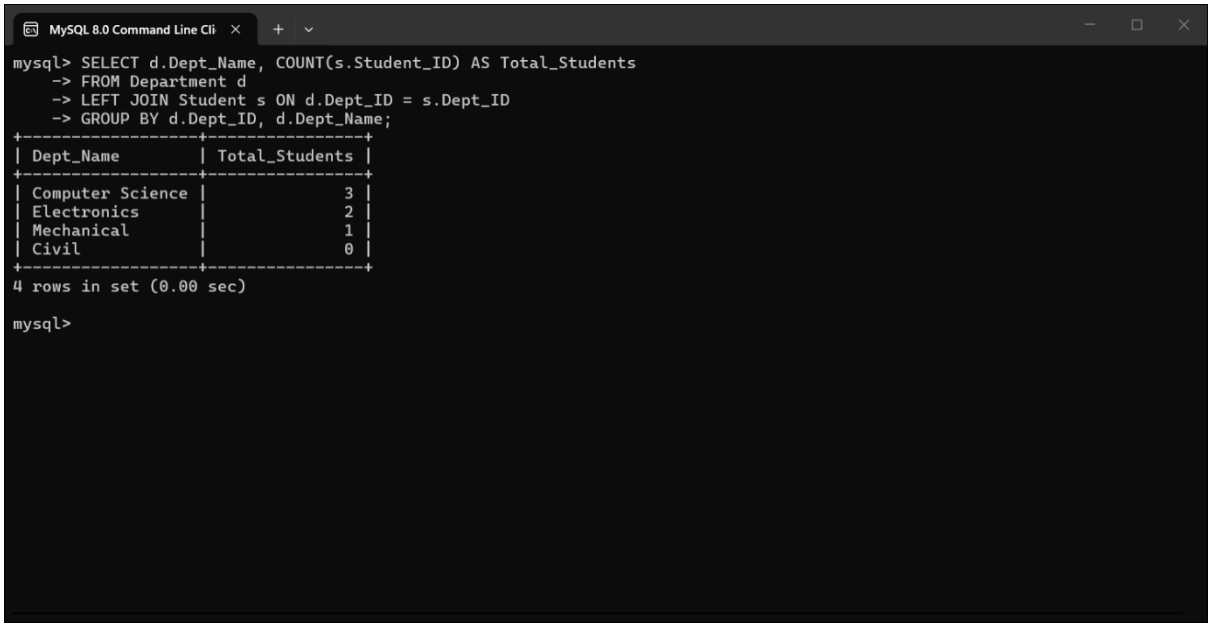
21. Find employees whose salary is greater than the average salary of their department using joins/subqueries.



22. Find the department having the maximum number of students using joins and aggregation.



23. Display departments and the total number of students in each department.



24. Display courses along with the number of students enrolled in each course

```
MySQL 8.0 Command Line Cli

mysql> SELECT c.Course_Name, COUNT(s.Student_ID) AS Total_Students
    -> FROM Course c
    -> LEFT JOIN Student s ON c.Course_ID = s.Course_ID
    -> GROUP BY c.Course_ID, c.Course_Name;
+-------------+----------------+
| Course_Name | Total_Students |
+-------------+----------------+
| B.Tech      |              4 |
| BCA         |              2 |
| MCA         |              1 |
| M.Tech      |              0 |
+-------------+----------------+
4 rows in set (0.00 sec)

mysql>
```

25. Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions to generate a student-department-course report.

```
MySQL 8.0 Command Line Cli

mysql> SELECT d.Dept_Name,
    ->        IFNULL(c.Course_Name, 'Not Assigned') AS Course_Name,
    ->        COUNT(s.Student_ID)  AS Total_Students,
    ->        ROUND(AVG(s.Age), 1) AS Average_Age,
    ->        MIN(s.Age)           AS Youngest,
    ->        MAX(s.Age)           AS Oldest
    -> FROM Student s
    -> INNER JOIN Department d ON s.Dept_ID   = d.Dept_ID
    -> LEFT JOIN  Course c     ON s.Course_ID = c.Course_ID
    -> GROUP BY d.Dept_Name, c.Course_Name
    -> ORDER BY d.Dept_Name, Course_Name;
+------------------+-------------+----------------+-------------+----------+--------+
| Dept_Name        | Course_Name | Total_Students | Average_Age | Youngest | Oldest |
+------------------+-------------+----------------+-------------+----------+--------+
| Computer Science | B.Tech      |              2 |        20.0 |       20 |     20 |
| Computer Science | BCA         |              1 |        19.0 |       19 |     19 |
| Electronics      | B.Tech      |              1 |        21.0 |       21 |     21 |
| Electronics      | MCA         |              1 |        19.0 |       19 |     19 |
| Mechanical       | B.Tech      |              1 |        22.0 |       22 |     22 |
+------------------+-------------+----------------+-------------+----------+--------+
5 rows in set (0.00 sec)

mysql>
```